

ArrayList Collection Class in C# with Examples

In this article, I am going to discuss the **Non-Generic ArrayList Collection Class in C#** with Examples. Please read our previous article before proceeding to this article where we discussed the [Introduction to Collections in C#](#). ArrayList is a powerful feature of the C# language. It is the non-generic type of collection that is defined in the **System.Collections** namespace. At the end of this article, you will understand the following pointers.

1. [What is ArrayList in C#?](#)
2. [How to Create an ArrayList in C# with Examples?](#)
3. [How to Add Elements into ArrayList in C#?](#)
4. [How to Access an ArrayList in C# with Examples?](#)
5. [How to Iterate an ArrayList in C#?](#)
6. [How to Insert an Element into a Specified Position in an ArrayList Collection in C#?](#)
7. [How to Remove Elements from ArrayList in C# with Examples?](#)
8. [How to Remove all Elements from the ArrayList in C#?](#)
9. [How do we Check whether an Element exists in ArrayList or not in C#?](#)
10. [How to Clone the Non-Generic ArrayList Collection in C#?](#)
11. [How to Copy an ArrayList to an Existing Array in C#?](#)
12. [How to Sort the Elements of an ArrayList Collection in C#?](#)
13. [What is the difference between an Array and an ArrayList in C#?](#)

What is ArrayList in C#?

The ArrayList in C# is a non-generic collection class that works like an array but provides the facilities such as dynamic resizing, adding, and deleting elements from the middle of a collection. The ArrayList in C# can be used to add unknown data i.e. when we don't know the types of data and size of the data, then we can use ArrayList.

It is used to create a dynamic array means the size of the array is increase or decreases automatically according to the requirement of our program. There is no need to specify the size of the ArrayList. In ArrayList, we can store elements of the same type and of different types.

Properties of ArrayList Class in C#:

1. The Elements can be added and removed from the Array List collection at any point in time.
2. The ArrayList is not guaranteed to be sorted.
3. The capacity of an ArrayList is the number of elements the ArrayList can hold.
4. Elements in this collection can be accessed using an integer index. Indexes in this collection are zero-based.
5. It allows duplicate elements.

How to Create an ArrayList in C#?

The ArrayList in C# provides the following three constructors which we can use to create an instance of the ArrayList class.

1. **ArrayList():** The method is used to initialize a new instance of the ArrayList class that is empty and has the default initial capacity.
2. **ArrayList(Collection c):** The method is used to initialize a new instance of the ArrayList class that contains elements copied from the specified collection and that have the same initial capacity as the number of elements copied. The parameter c specifies the Collection whose elements are copied to the new list.
3. **ArrayList(int capacity):** The method is used to initialize a new instance of the ArrayList class that is empty and has the specified initial capacity. The parameter capacity specifies the number of elements that the new list can initially store.

First, we need to import the System.Collections namespace and then we can create an instance of ArrayList by using the first constructor as follows. You can use any of the following syntaxes,

```
ArrayList arrayList = new ArrayList();
```

```
// or
```

```
var arrayList = new ArrayList();
```

How to Add Elements into ArrayList in C#?

The ArrayList non-generic class provides the Add() method which we can use to add elements to the array list or even we can use the object initializer syntax to add elements in an ArrayList. The most important point is that we can add multiple different types of elements in an ArrayList even though it is also possible to add duplicate elements.

Let us see an example to understand both approaches to adding elements in a collection of type ArrayList in C#. Please have a look at the below example. Here, you can observe, that we have added different types of data as well as duplicate data and it is accepted.

```
using System;
using System.Collections;
namespace Csharp8Features
{
    public class ArrayListDemo
    {
        public static void Main()
        {
            //Adding elements to ArrayList using Add() method
            ArrayList arrayList1 = new ArrayList();
            arrayList1.Add(101); //Adding Integer Value
            arrayList1.Add("James"); //Adding String Value
            arrayList1.Add("James"); //Adding Duplicate Value
            arrayList1.Add(" "); //Adding Empty
            arrayList1.Add(true); //Adding Boolean
            arrayList1.Add(4.5); //Adding double
            arrayList1.Add(null); //Adding null
            foreach (var item in arrayList1)
            {
                Console.WriteLine(item);
            }
            //Adding Elements to ArrayList using object initializer syntax
            var arrayList2 = new ArrayList()
            {
```

```

        102, "Smith", "Smith", true, 15.6
    };
    foreach (var item in arrayList2)
    {
        Console.WriteLine(item);
    }
    Console.ReadKey();
}
}
}

```

Output:

```

101
James
James

True
4.5

102
Smith
Smith
True
15.6

```

How to Access an ArrayList in C#?

If you go to the definition of ArrayList, then you will see that the ArrayList class implements the IList interface as shown in the below image. As it implements the IList interface, so we can access the elements using an indexer, in the same way as an array. The index starts from zero and increases by one for each subsequent element. So, when a collection class implements the IList interface, then we can access the elements of that collection by using the integer indexes.

```

//
// Summary:
//     Implements the System.Collections.IList interface using an array whose size is
//     dynamically increased as required.
[DefaultMember("Item")]
public class ArrayList : ICollection, IEnumerable, IList, ICloneable
{

```

While adding the elements into ArrayList, it will automatically cast the elements into object types and then store them in the collection. So, while accessing the elements an explicit casting to the appropriate types is required, or else you can use the var variable. For a better understanding, please have a look at the below example. The code is self-explained, please go through the comments.

```
using System;
```

```

using System.Collections;
namespace Csharp8Features
{
    public class ArrayListDemo
    {
        public static void Main()
        {
            //Adding elements to ArrayList using Add() method
            ArrayList arrayList1 = new ArrayList();
            arrayList1.Add(101); //Adding Integer Value
            arrayList1.Add("James"); //Adding String Value
            arrayList1.Add(true); //Adding Boolean
            arrayList1.Add(4.5); //Adding double
            //Accessing individual elements from ArrayList using Indexer
            int firstElement = (int)arrayList1[0]; //returns 101
            string secondElement = (string)arrayList1[1]; //returns "James"
            //int secondElement = (int) arrayList1[1]; //Error: cannot cover s
int
            Console.WriteLine($"First Element: {firstElement}, Second Element:
{secondElement}");
            //Using var keyword without explicit casting
            var firstItem = arrayList1[0]; //returns 101
            var secondItem = arrayList1[1]; //returns "James"
            //var fifthElement = arrayList1[5]; //Error: Index out of range
            Console.WriteLine($"First Item: {firstItem}, Second Item: {secondItem}");
            //update elements
            arrayList1[0] = "Smith";
            arrayList1[1] = 1010;
            //arrayList1[5] = 500; //Error: Index out of range
            foreach (var item in arrayList1)
            {
                Console.Write($"{item} ");
            }
            Console.ReadKey();
        }
    }
}

```

Output:

```

First Element: 101, Second Element: James
First Item: 101, Second Item: James
Smith 1010 True 4.5

```

How to Iterate an ArrayList in C#?

If you go to the definition of ArrayList, then you will also see that the ArrayList non-generic collection class implements the ICollection interface as shown in the below image. And we know the ICollection interface supports iteration of the collection types. So, we can either use the foreach loop and for loop to iterate a collection of type ArrayList.

```
//
// Summary:
//     Implements the System.Collections.IList interface using an array whose size is
//     dynamically increased as required.
[DefaultMember("Item")]
public class ArrayList : ICollection, IEnumerable, IList, ICloneable
{
```

The Count property of ArrayList returns the total number of elements present in an ArrayList. Let us understand this with an example.

```
using System;
using System.Collections;
namespace Csharp8Features
{
    public class ArrayListDemo
    {
        public static void Main()
        {
            //Adding elements to ArrayList using Add() method
            ArrayList arrayList1 = new ArrayList();
            arrayList1.Add(101); //Adding Integer Value
            arrayList1.Add("James"); //Adding String Value
            arrayList1.Add(true); //Adding Boolean
            arrayList1.Add(4.5); //Adding double
            //Iterating through foreach loop
            Console.WriteLine("Using ForEach Loop");
            foreach (var item in arrayList1)
            {
                Console.Write($"{item} ");
            }
            //Iterating through for loop
            Console.WriteLine("\n\nUsing For Loop");
            for (int i = 0; i < arrayList1.Count; i++)
            {
                Console.Write($"{arrayList1[i]} ");
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
Using ForEach Loop
101 James True 4.5

Using For Loop
101 James True 4.5
```

How to Insert an Element into a Specified Position in an ArrayList Collection in C#?

The Add method will add the element at the end of the collection i.e. after the last element. But, if you want to add the element at a specified position, then you need to use the Insert() method of the ArrayList class which will insert an element into the collection at the specified index position. The syntax to use the Insert method is given below.

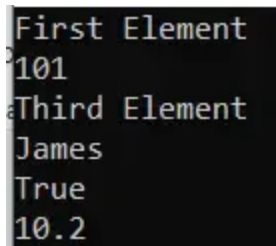
void Insert(int index, object? value);

Here, the parameter index specifies the index position at which the value should be inserted and the parameter value specifies the object to insert into the list. It is based on a zero index. For a better understanding, please have a look at the below example.

```
using System;
using System.Collections;
namespace Csharp8Features
{
    public class ArrayListDemo
    {
        public static void Main()
        {
            ArrayList arrayList = new ArrayList()
            {
                101,
                "James",
                true,
                10.20
            };

            //Insert "First Element" at First Position i.e. Index 0
            arrayList.Insert(0, "First Element");
            //Insert "Third Element" at Third Position i.e. Index 2
            arrayList.Insert(2, "Third Element");
            //Iterating through foreach loop
            foreach (var item in arrayList)
            {
                Console.WriteLine($"{item}");
            }
            Console.ReadKey();
        }
    }
}
```

Output:



```
First Element
101
Third Element
James
True
10.2
```

If we have a collection and we want to insert that collection into another collection of Array List, then we can use the InsertRange() method. The InsertRange() method Inserts the elements of a collection into the ArrayList at the specified index. The syntax is given below.

void InsertRange(int index, ICollection c)

Here, the parameter index specifies at which location the new elements should be inserted and the parameter c specifies the Collection whose elements should be inserted into the ArrayList. The collection itself cannot be null, but it can contain elements that are null. For a better understanding, please have a look at the below example.

```
using System;
using System.Collections;
namespace Csharp8Features
{
    public class ArrayListDemo
    {
        public static void Main()
        {
            ArrayList arrayList1 = new ArrayList()
            {
                "India",
                "USA",
                "UK",
                "Nepal"
            };
            Console.WriteLine("Array List Elements");
            foreach (var item in arrayList1)
            {
                Console.Write($"{item} ");
            }
            ArrayList arrayList2 = new ArrayList()
            {
                "Srilanka",
                "Japan",
                "Briterm"
            };
            arrayList1.InsertRange(2, arrayList2);

            Console.WriteLine("\n\nArray List Elements After InsertRange");
            foreach (var item in arrayList1)
            {
                Console.Write($"{item} ");
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
Array List Elements
```

```
India USA UK Nepal
```

```
Array List Elements After InsertRange
```

```
India USA Srilanka Japan Briterm UK Nepal
```

How to Remove Elements from ArrayList in C#?

If you want to remove elements from ArrayList in C#, then you can use Remove(), RemoveAt(), or RemoveRange() methods of the ArrayList class in C#.

1. **Remove(object? obj):** This method is used to remove the first occurrence of a specific object from the System.Collections.ArrayList. The parameter obj specifies the Object to remove from the ArrayList. The value can be null.
2. **RemoveAt(int index):** This method is used to remove the element at the specified index of the ArrayList. The parameter index specifies the index position of the element to remove.
3. **RemoveRange(int index, int count):** This method is used to remove a range of elements from the ArrayList. The parameter index specifies the starting index position of the range of elements to remove and the parameter count specifies the number of elements to remove.

For a better understanding, please have a look at the below example.

```
using System;
using System.Collections;
namespace Csharp8Features
{
    public class ArrayListDemo
    {
        public static void Main()
        {
            ArrayList arrayList = new ArrayList()
            {
                "India",
                "USA",
                "UK",
                "Nepal",
                "HongKong",
                "Srilanka",
                "Japan",
                "Briterm",
                "HongKong",
            };
            Console.WriteLine("Array List Elements");
            foreach (var item in arrayList)
            {
                Console.Write($"{item} ");
            }
            arrayList.Remove("HongKong"); //Removes first occurrence of null
        }
    }
}
```

```

        Console.WriteLine("\n\nArray List Elements After Removing First Occurrence of
HongKong");
        foreach (var item in arrayList)
        {
            Console.Write($"{item} ");
        }
        arrayList.RemoveAt(3); //Removes element at index position 3, it is at
index 3
        Console.WriteLine("\n\nArray List1 Elements After Removing Element from
index 3");
        foreach (var item in arrayList)
        {
            Console.Write($"{item} ");
        }
        arrayList.RemoveRange(0, 2); //Removes two elements starting from index 0
index)
        Console.WriteLine("\n\nArray List Elements After Removing First Two
Elements");
        foreach (var item in arrayList)
        {
            Console.Write($"{item} ");
        }
        Console.ReadKey();
    }
}
}

```

Output:

```

Array List Elements
India USA UK Nepal HongKong Srilanka Japan Briter HongKong

Array List Elements After Removing First Occurances of HongKong
India USA UK Nepal Srilanka Japan Briter HongKong

Array List1 Elements After Removing Element from Index 3
India USA UK Srilanka Japan Briter HongKong

Array List Elements After Removing First Two Elements
UK Srilanka Japan Briter HongKong

```

How to Remove all the elements from the ArrayList in C#?

If you want to remove all elements or clear all the elements from the ArrayList, then you can use the Clear() method of the ArrayList class but this method doesn't reduce the capacity of the ArrayList. Let us see an example for a better understanding.

```

using System;
using System.Collections;

```

```

namespace Csharp8Features
{
    public class ArrayListDemo
    {
        public static void Main()
        {
            ArrayList arrayList = new ArrayList()
            {
                "India",
                "USA",
                "UK",
                "Denmark",
                "Nepal",
            };
            int totalItems = arrayList.Count;
            Console.WriteLine(string.Format($"Total Items: {totalItems}, Capacity: {arrayList.Capacity}"));
            //Remove all items from the Array list
            arrayList.Clear();
            totalItems = arrayList.Count;
            Console.WriteLine(string.Format($"Total Items After Clear(): {totalItems}, Capacity: {arrayList.Capacity}"));
            Console.ReadKey();
        }
    }
}

```

Output:

```

Total Items: 5, Capacity: 8
Total Items After Clear(): 0, Capacity: 8

```

How do we Check whether an Element exists in ArrayList or not in C#?

In order to check whether an element exists or not in ArrayList, we need to use the Contains() method of the ArrayList non-generic collection class in C#. It returns true if exists otherwise returns false. The following is the syntax to use the Contains() method.

1. **bool Contains(object? item):** This method is used to determine whether an element is in the ArrayList. The parameter item specifies the Object to locate in the ArrayList. The value can be null. It returns true if the item is found in the ArrayList; otherwise, false.

For a better understanding, please have a look at the below example.

```

using System;
using System.Collections;
namespace Csharp8Features
{
    public class ArrayListDemo

```

```

{
    public static void Main()
    {
        ArrayList arrayList = new ArrayList()
        {
            "India",
            "UK",
            "Nepal",
            101
        };
        Console.WriteLine("Array List Elements");
        foreach (var item in arrayList)
        {
            Console.Write($"{item} ");
        }
        Console.WriteLine($"\n\nIs ArrayList Contains India:
{arrayList.Contains("India")}"); // true
        Console.WriteLine($"Is ArrayList Contains USA: {arrayList.Contains(
// false
        Console.WriteLine($"Is ArrayList Contains 101: {arrayList.Contains(
// true
        Console.WriteLine($"Is ArrayList Contains 10.5: {arrayList.Contain
// false
        Console.ReadKey();
    }
}
}

```

Output:

```

Array List Elements
India UK Nepal 101

Is ArrayList Contains India: True
Is ArrayList Contains USA: False
Is ArrayList Contains 101: True
Is ArrayList Contains 10.5: False

```

Note: It is not recommended to use the non-generic collection class `ArrayList` in C# due to performance issues i.e. boxing and unboxing as it is operating on the object data type. So, instead of using `ArrayList`, it is recommended to use the generic collection `List<object>` to store heterogeneous objects. To store data of the same data type, use Generic `List<T>`.

How to Clone the Non-Generic ArrayList Collection in C#?

If you want to clone the Non-Generic `ArrayList` collection in C#, then you need to use the following `Clone()` method provided by the `ArrayList` Collection Class.

1. **Clone():** This method is used to create and return a shallow copy of the `ArrayList`.

For a better understanding, please have a look at the below example.

```
using System;
using System.Collections;
namespace Csharp8Features
{
    public class ArrayListDemo
    {
        public static void Main()
        {
            ArrayList arrayList = new ArrayList()
            {
                "India",
                "USA",
                "UK",
                "Denmark",
                "HongKong",
            };
            Console.WriteLine("Array List Elements:");
            foreach (var item in arrayList)
            {
                Console.WriteLine($"{item} ");
            }
            //Creating a clone of the arrayList using Clone method
            ArrayList cloneArrayList = (ArrayList)arrayList.Clone();
            Console.WriteLine("\nCloned ArrayList Elements:");
            foreach (var item in cloneArrayList)
            {
                Console.WriteLine($"{item} ");
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
Array List Elements:
India
USA
UK
Denmark
HongKong

Cloned ArrayList Elements:
India
USA
UK
Denmark
HongKong
```

How to copy an ArrayList to an existing array in C#?

In order to copy an ArrayList to an existing array in C#, the Non-Generic ArrayList Collection Class provides the following three methods.

1. **CopyTo(Array array):** This method is used to copy the entire ArrayList to a compatible one-dimensional Array, starting at the beginning of the target array. The parameter array specifies the one-dimensional Array that is the destination of the elements copied from ArrayList. The Array must have zero-based indexing. If the parameter array is null, then it will throw `ArgumentNullException`.
2. **CopyTo(Array array, int arrayIndex):** This method is used to copy the entire ArrayList to a compatible one-dimensional Array, starting at the specified index of the target array. Here, the parameter array specifies the one-dimensional array that is the destination of the elements copied from the ArrayList. The Array must have zero-based indexing. The `arrayIndex` parameter specifies the zero-based index in the array at which copying begins. If the parameter array is null, then it will throw `ArgumentNullException`. If the parameter `arrayIndex` is less than zero, then it will throw `ArgumentOutOfRangeException`.
3. **CopyTo(int index, Array array, int arrayIndex, int count):** This method is used to copy a range of elements from the `System.Collections.ArrayList` to a compatible one-dimensional Array, starting at the specified index of the target array. The `index` parameter specifies the zero-based index in the source `System.Collections.ArrayList` at which copying begins. The `array` parameter specifies the one-dimensional Array that is the destination of the elements copied from ArrayList. The Array must have zero-based indexing. The `arrayIndex` parameter specifies the zero-based index in the array at which copying begins. The `count` parameter specifies the number of elements to copy. If the parameter array is null, then it will throw `ArgumentNullException`. If the parameter `index` is less than zero, `arrayIndex` is less than zero, or the `count` is less than zero, then it will throw `ArgumentOutOfRangeException`.

Let us see an example for a better understanding.

```
using System;
using System.Collections;
namespace Csharp8Features
{
    public class ArrayListDemo
    {
        public static void Main()
        {
```

```

ArrayList arrayList = new ArrayList()
{
    "India",
    "USA",
    "UK",
    "Denmark",
    "HongKong",
};
Console.WriteLine("Array List Elements:");
foreach (var item in arrayList)
{
    Console.WriteLine($"{item} ");
}
//Copying the arrayList to an object array
object[] arrayListCopy = new object[arrayList.Count];
arrayList.CopyTo(arrayListCopy);
Console.WriteLine("\nArrayList Copy Array Elements:");
foreach (var item in arrayListCopy)
{
    Console.WriteLine(item);
}
Console.ReadKey();
}
}
}

```

Output:

```

Array List Elements:
India
USA
UK
Denmark
HongKong

ArrayList Copy Array Elements:
India
USA
UK
Denmark
HongKong

```

How to Sort the Elements of an ArrayList Collection in C#?

If you want to sort the elements of the ArrayList in C#, then you can use the following Sort() method of the ArrayList class.

1. **Sort():** This method is used to sort the elements in the entire System.Collections.ArrayList.

2. **Sort(IComparer? comparer):** This method is used to sort the elements in the entire ArrayList using the specified comparer.
3. **Sort(int index, int count, IComparer? comparer):** This method is used to sort the elements in a range of elements in ArrayList using the specified comparer.

These methods use the QuickSort algorithm to perform sorting on the ArrayList and the elements are arranged in ascending order. For a better understanding, please have a look at the below example.

```
using System;
using System.Collections;
namespace Csharp8Features
{
    public class ArrayListDemo
    {
        public static void Main()
        {
            ArrayList arrayList = new ArrayList()
            {
                "India",
                "USA",
                "UK",
                "Denmark",
                "Nepal",
                "HongKong",
                "Australilla",
                "Srilanka",
                "Japan",
                "Briterm",
                "Brazil",
            };
            Console.WriteLine("Array List Elements Before Sorting");
            foreach (var item in arrayList)
            {
                Console.Write($"{item} ");
            }
            // Sorting the elements of ArrayList Using sort() method
            arrayList.Sort();
            Console.WriteLine("\n\nArray List Elements After Sorting");
            foreach (var item in arrayList)
            {
                Console.Write($"{item} ");
            }
            Console.ReadKey();
        }
    }
}
```

Output:

Array List Elements Before Sorting

India USA UK Denmark Nepal HongKong Austraila Srilanka Japan Briterm Brazil

Array List Elements After Sorting

Austraila Brazil Briterm Denmark HongKong India Japan Nepal Srilanka UK USA

What is the difference between an Array and an Array List in C#?

The ArrayList collection in C# is very much similar to the Arrays data type. The major difference between them is the dynamic nature of the non-generic collection ArrayList. For arrays, we need to define the size i.e. the number of elements that the array can hold at the time of array declaration. But in the case of the ArrayList collection in C#, this does not need to be done beforehand. Elements can be added or removed from the Array List collection at any point in time.

This is one of the frequently asked interview questions in C#. So let us discuss the difference between an array and an ArrayList.

Array:

1. Fixed Length
2. Cannot insert it into the middle
3. Cannot delete from middle
4. It is type-safe, so we can store only similar types of data based on the data type.
5. Boxing and Unboxing are not required.

ArrayList:

1. Variable Length
2. Can insert an element into the middle of the collection
3. Can delete elements from the middle of the collection
4. It is not type-safe, so we can store any type of data.
5. Boxing and Unboxing are required as it is operated on the object data type.

In the next article, I am going to discuss [Non-Generic Hashtable Collection Class in C#](#) with Examples. Here, in this article, I try to explain the **Non-Generic ArrayList Collection Class in C#** with Examples. I hope you enjoy this Non-Generic ArrayList Collection Class in C# with Examples article. I would like to have your feedback. Please post your feedback, question, or comments about this article.